

UNIVERSIDADE DE MOGI DAS CRUZES

Luiza Helena Tardelli Marçulli Espíndola

RELATÓRIO - TICKETHUB

**Mogi das Cruzes - SP
2026**

UNIVERSIDADE DE MOGI DAS CRUZES

Luiza Helena Tardelli Marçulli Espíndola

RELATÓRIO - TICKETHUB

Projeto apresentado à Universidade de Mogi das Cruzes, como requisito para a disciplina de Paradigmas de Orientação a Objetos.

Prof.(a) Orientador: Alessandro Aparecido da Silva Horas.

**Mogi das Cruzes - SP
2026**

SUMÁRIO

1	INTRODUÇÃO.....	4
2	ARQUITETURA GERAL DO SISTEMA.....	4
3	MODELAGEM DAS ENTIDADES DE DOMÍNIO	5
	3.1 Classe Abstrata Ingresso.....	5
	3.2 Subtipos de Ingresso.....	6
	3.3 Entidade Evento.....	7
4	IMPLEMENTAÇÃO DO POLIMORFISMO	7
5	PADRÃO MVC E SEPARAÇÃO DE RESPONSABILIDADES	8
6	DECISÕES DE PERSISTÊNCIA	9
7	INTERFACE	9
8	CONSIDERAÇÕES FINAIS	10
9	REFERÊNCIAS	10

1 INTRODUÇÃO

O presente relatório tem como finalidade descrever as decisões de modelagem orientada a objetos e de implementação adotadas no desenvolvimento do sistema *TicketHub*, uma aplicação web destinada ao controle e à emissão de ingressos para eventos.

O sistema foi desenvolvido utilizando Java 17, Spring Boot 3.2, *Thymeleaf* e MongoDB Atlas, com a aplicação prática dos princípios da Programação Orientada a Objetos (POO), com destaque para abstração, herança e polimorfismo.

O objetivo central deste trabalho consiste em demonstrar como o uso do polimorfismo possibilita o tratamento uniforme de objetos de diferentes tipos, contribuindo para a redução do acoplamento e para o aumento da extensibilidade do sistema. Para isso, foi estruturada uma hierarquia de classes baseada na classe abstrata *Ingresso*, da qual derivam três especializações: *IngressoNormal*, *IngressoVIP* e *IngressoMeia*.

2 ARQUITETURA GERAL DO SISTEMA

O sistema *TicketHub* foi estruturado com base no padrão arquitetural MVC (*Model-View-Controller*), organizado em três camadas principais, acrescidas de uma camada de serviços responsável pelo encapsulamento das regras de negócio.

Segundo a DevMedia (2013), o modelo (*Model*) é responsável pelos dados e regras de negócio, a visão (*View*) pela apresentação das informações ao usuário e o controlador (*Controller*) pela intermediação entre essas camadas, recebendo as requisições e acionando o processamento adequado.

A adoção dessa divisão teve como objetivo principal garantir a separação de responsabilidades, evitando que regras de negócio fossem implementadas indevidamente nas camadas de controle ou de apresentação.

A escolha do Spring Boot como framework principal justifica-se pela sua capacidade de acelerar o desenvolvimento, especialmente por meio de recursos como configuração automática, injeção de dependências e integração simplificada com bancos de dados.

Além disso, o uso do Spring Data MongoDB permite a abstração da camada de persistência, eliminando a necessidade de escrita manual de consultas e possibilitando a definição de repositórios por meio de interfaces.

Tabela 01 – Camadas da Arquitetura

Camada	Pacote	Responsabilidade
Model	com.ingressos.model	Entidades de domínio: Ingresso (abstrata), subtipos e Evento
Repository	com.ingressos.repository	Acesso ao MongoDB via Spring Data (interfaces)
Service	com.ingressos.service	Regras de negócio, fábrica polimórfica de ingressos
Controller	com.ingressos.controller	Mapeamento HTTP, delegação para services, população das views
View	resources/templates	Templates Thymeleaf renderizados no servidor
Config	com.ingressos.config	DataInitializer, WebConfig, GlobalExceptionHandler

Fonte: Autora (2026).

3 MODELAGEM DAS ENTIDADES DE DOMÍNIO

3.1 Classe Abstrata Ingresso

Uma das decisões mais relevantes no processo de modelagem foi a definição da classe Ingresso como abstrata. Essa escolha foi fundamentada nos seguintes aspectos:

- Os diferentes tipos de ingresso compartilham diversos atributos em comum, como identificador, dados do comprador e informações do evento;
- Cada subtipo possui uma regra específica de cálculo de valor, o que exige a definição de métodos abstratos;
- Há necessidade de reutilização de comportamentos comuns entre os subtipos.

Dessa forma, a classe abstrata define um contrato polimórfico, garantindo que todas as subclasses implementem os seguintes métodos:

- cálculo do valor;
- impressão do ingresso;
- descrição do tipo.

Essa abordagem favorece a extensibilidade do sistema, permitindo a inclusão de novos tipos de ingresso sem a necessidade de alterações nas demais camadas.

3.2 Subtipos de Ingresso

Cada subtipo implementa sua própria regra de precificação:

- IngressoNormal: mantém o valor base;
- IngressoVIP: aplica um multiplicador sobre o valor base;
- IngressoMeia: aplica um desconto proporcional.

A utilização de constantes imutáveis (`static final`) para representar multiplicadores e descontos constitui uma boa prática, pois centraliza a lógica de cálculo e facilita futuras alterações.

Além disso, cada subtipo possui atributos específicos, como benefícios no ingresso VIP e o motivo do desconto no ingresso meia-entrada, respeitando o princípio de especialização.

3.3 Entidade Evento

A classe Evento foi modelada de forma independente da hierarquia de ingressos, respeitando o Princípio da Responsabilidade Única (*Single Responsibility Principle* – SRP).

De acordo com a DevMedia (2010), o SRP estabelece que uma classe deve ser responsável por apenas uma parte da funcionalidade do sistema, possuindo, portanto, um único motivo para sofrer alterações. Isso implica que diferentes responsabilidades devem ser separadas em diferentes classes, evitando o acúmulo de funções distintas em uma única estrutura.

Nesse contexto, a separação entre as classes Evento e Ingresso segue diretamente esse princípio. A classe Evento é responsável exclusivamente pelo gerenciamento do evento, incluindo aspectos como capacidade e controle de vagas disponíveis. Por outro lado, a classe Ingresso concentra as informações relacionadas ao comprador e à lógica de cálculo do valor.

Segundo a mesma abordagem, quando uma classe assume múltiplas responsabilidades, ela se torna mais difícil de manter, compreender e modificar, além de aumentar o acoplamento entre partes do sistema. Dessa forma, a divisão clara de responsabilidades contribui para um código mais organizado, modular e de fácil evolução.

Além disso, a implementação de métodos como `temVagasDisponiveis()` e `registrarVenda()` diretamente na entidade Evento garante que as regras de negócio estejam devidamente encapsuladas, evitando duplicações e assegurando maior consistência no sistema.

4 IMPLEMENTAÇÃO DO POLIMORFISMO

O polimorfismo é um dos conceitos centrais aplicados no desenvolvimento do sistema *TicketHub*, estando presente tanto na camada de serviço quanto na camada de apresentação. No sistema, diferentes tipos de ingressos (`IngressoNormal`, `IngressoVIP`, `IngressoMeia`) são

tratados de maneira uniforme por meio da superclasse Ingresso, permitindo que o código opere de forma genérica, independentemente do tipo concreto do objeto.

De acordo com a DevMedia (2010), o polimorfismo consiste na capacidade de métodos com a mesma assinatura apresentarem comportamentos distintos, dependendo do objeto que os executa. Esse comportamento é definido nas subclasses e resolvido dinamicamente em tempo de execução, sem a necessidade de verificação explícita do tipo do objeto.

No contexto do *TicketHub*, esse conceito é aplicado no processo de emissão de ingressos, no qual diferentes objetos são instanciados conforme o tipo selecionado, mas posteriormente manipulados de forma genérica como instâncias da classe Ingresso. Assim, métodos como `calcularValor()` e `descricaoTipo()` podem ser invocados sem a necessidade de estruturas condicionais, sendo automaticamente direcionados para a implementação correspondente ao subtipo específico.

Esse mesmo comportamento se estende à camada de apresentação, onde os dados dos ingressos são exibidos ao usuário sem que seja necessário distinguir explicitamente entre os diferentes tipos existentes. Dessa forma, a aplicação mantém um código mais limpo, organizado e de fácil manutenção.

Ainda conforme a DevMedia (2010), o uso do polimorfismo contribui para a reutilização de código e para a flexibilidade do sistema, permitindo que novas funcionalidades sejam incorporadas com menor impacto nas estruturas já existentes. No caso do *TicketHub*, isso possibilita a adição de novos tipos de ingresso sem a necessidade de modificar as demais camadas da aplicação, evidenciando uma arquitetura extensível e alinhada com boas práticas de desenvolvimento orientado a objetos.

5 PADRÃO MVC E SEPARAÇÃO DE RESPONSABILIDADES

Como já descrito anteriormente, o sistema *TicketHub* foi desenvolvido com base no padrão arquitetural MVC (*Model-View-Controller*).

Os *controllers* foram implementados de forma enxuta, limitando-se ao tratamento de requisições HTTP e delegação para a camada de serviços.

A camada de serviços concentra toda a lógica de negócio, garantindo maior organização e reutilização de código.

Já o tratamento de exceções foi centralizado em um manipulador global, o que evita repetição de código e melhora a experiência do usuário ao apresentar mensagens de erro mais claras.

Dessa forma, a adoção do padrão MVC contribui para uma arquitetura mais modular, organizada e alinhada com boas práticas de desenvolvimento.

6 DECISÕES DE PERSISTÊNCIA

A escolha do MongoDB se justifica pela natureza dos dados manipulados, que são estruturados de forma independente e não exigem relacionamentos complexos.

O modelo orientado a documentos se adapta bem à hierarquia de classes utilizada, permitindo que cada tipo de ingresso armazene apenas os atributos relevantes.

O uso do Spring Data MongoDB simplifica ainda mais a persistência, permitindo a definição de consultas por convenção de nomenclatura.

7 INTERFACE

A interface do sistema foi projetada com foco na clareza e usabilidade.

A seleção do tipo de ingresso é feita por meio de elementos visuais interativos, permitindo ao usuário visualizar previamente os valores calculados.

Entretanto, o cálculo definitivo é sempre realizado no servidor, garantindo a integridade das regras de negócio.

Além disso, o sistema apresenta telas detalhadas e dashboards com indicadores relevantes, utilizando o polimorfismo para cálculo de valores de forma transparente.

8 CONSIDERAÇÕES FINAIS

O desenvolvimento do *TicketHub* permitiu a aplicação prática dos conceitos fundamentais da Programação Orientada a Objetos em um contexto real.

A utilização de uma classe abstrata como base para a hierarquia de ingressos mostrou-se uma decisão eficiente, eliminando a necessidade de verificações condicionais e promovendo maior flexibilidade.

A integração entre Spring Boot e MongoDB contribuiu para uma arquitetura simples, organizada e escalável.

Como possibilidade de evolução, o sistema pode ser facilmente expandido com novos tipos de ingressos.

9 REFERÊNCIAS

DEVMEDIA. **Arquitetura: o princípio da responsabilidade única, 2010.** Disponível em: <<https://www.devmedia.com.br/arquitetura-o-principio-da-responsabilidade-unica/18700>>. Acesso em: abril 2026.

DEVMEDIA. **Conceitos e exemplos: polimorfismo na programação orientada a objetos, 2010.** Disponível em: <<https://www.devmedia.com.br/conceitos-e-exemplos-polimorfismo-programacao-orientada-a-objetos/18701>>. Acesso em: abril 2026.

DEVMEDIA. **Introdução ao padrão MVC, 2013.** Disponível em: <<https://www.devmedia.com.br/introducao-ao-padrao-mvc/29308>>. Acesso em: abril 2026.

MONGODB. **Integração com o Spring Data (Java Sync Driver).** Disponível em: <<https://www.mongodb.com/pt-br/docs/drivers/java/sync/current/integrations/spring-data-integration/>>. Acesso em: maio 2026.